Tecnologie Emergenti per la Trasmissione dei Dati e delle Informazioni

Prof. G.L. FORESTI, Prof. Andrea TOMA
Prof. Ivan SCAGNETTO

Dipartimento di Scienze Matematiche, Informatiche e Fisiche (DMIF)

Università degli Studi di Udine

DISCLAIMER

Il materiale contenuto in queste slide è riservato esclusivamente agli studenti del corso di Tecnologie Emergenti per la Trasmissione dei Dati e delle Informazioni del Corso di Studi in **TECNOLOGIE DIGITALI E COMUNICAZIONE PER LE INDUSTRIE CREATIVE** (classe L20&L31) dell'Università degli Studi di Udine.

E' vietato ogni utilizzo del materiale contenuto in queste slide diverso da quello inerente la preparazione dell'esame del suddetto corso, ed in particolare è espressamente vietato il suo utilizzo per qualsiasi scopo commerciale e/o di lucro.

LEZIONE 3



ASPETTI SOFTWARE DELLE RETI



PARTE I

Livelli, Servizi e Protocolli



• Per ridurre la complessità di progetto, le reti sono in generale organizzate a ***livelli (layer)***, ciascuno costruito sopra il precedente.

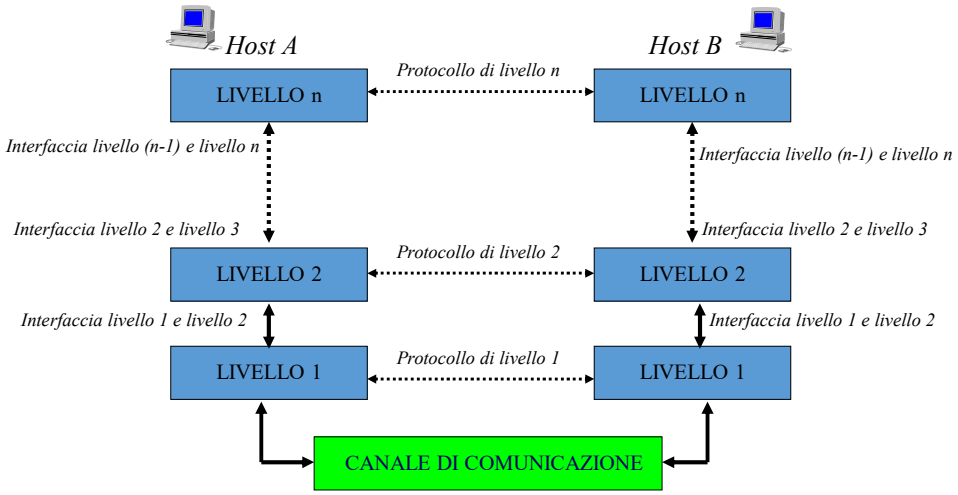
• Lo scopo di un livello è offrire certi ***servizi*** ai livelli più alti, nascondendo i dettagli su come tali servizi siano implementati.

• Il ***livello n*** su un host porta avanti una conversazione con il ***livello n*** su un altro host.

• Le regole e le convenzioni che governano la conversazione sono indicate col termine di ***protocolli (di livello n)***.

I protocolli





Architettura di rete



• L'insieme dei livelli e dei relativi protocolli è detto ***architettura di rete***.

• La specifica dell'architettura di rete deve essere sufficientemente dettagliata da consentire la realizzazione di SW e/o HW che, per ogni livello, rispettino il relativo protocollo.

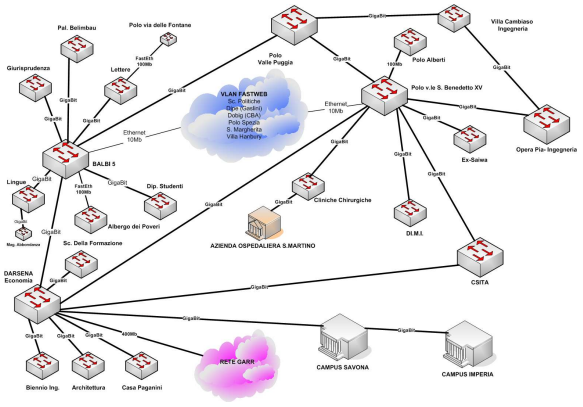
Tutti gli host devono contenere implementazioni conformi in termini di livelli e di protocolli

Architettura di rete (2)



Un'architettura di rete può essere:

- ***Proprietaria***
- ***Standard de facto***
- ***Standard de iure***



Architettura di rete proprietaria

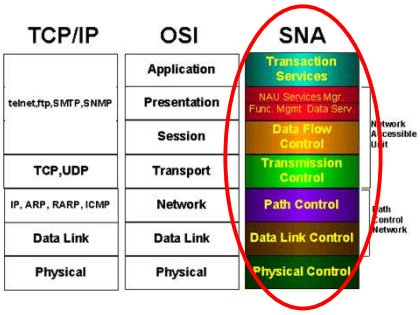


• Un'**architettura proprietaria** si basa su soluzioni definite da ogni singolo costruttore ➡ E' incompatibile con altri tipi di architetture.

• Le specifiche non sono pubbliche per impedire che si possano costruire apparati compatibili (livello di sicurezza elevato)

Esempi:

- **IBM SNA** (System Network Architecture)
- DEC (Digital Equipement Corporation)
- Novell IPX;



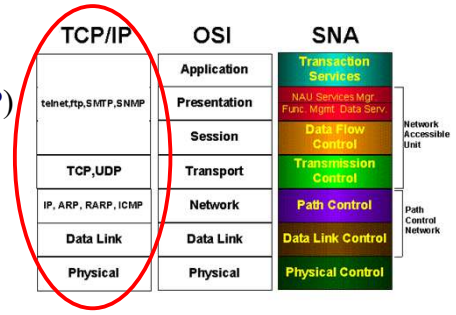
Architettura di rete standard de facto



• Un'**architettura standard de facto** è un'architettura basata su specifiche di pubblico dominio (per cui diversi costruttori possono proporre la propria implementazione).

Esempi:

- ***Internet Protocol Suite*** (detta anche architettura TCP/IP)
- ***Bluetooth*** (inizialmente sviluppato da Ericsson)

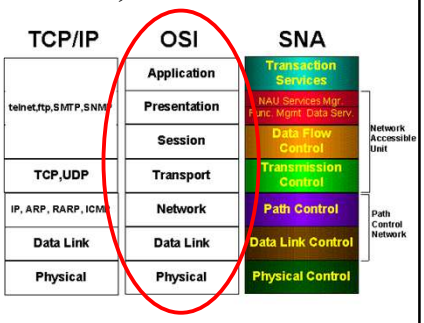


Architettura di rete standard de iure



• Un'**architettura standard de iure** è un'architettura basata su specifiche (di pubblico dominio) approvate da enti internazionali che si occupano di standardizzazione.

• Ogni costruttore può proporre una propria implementazione, ma le interfacce devono rispettare determinate specifiche.



Esempi:

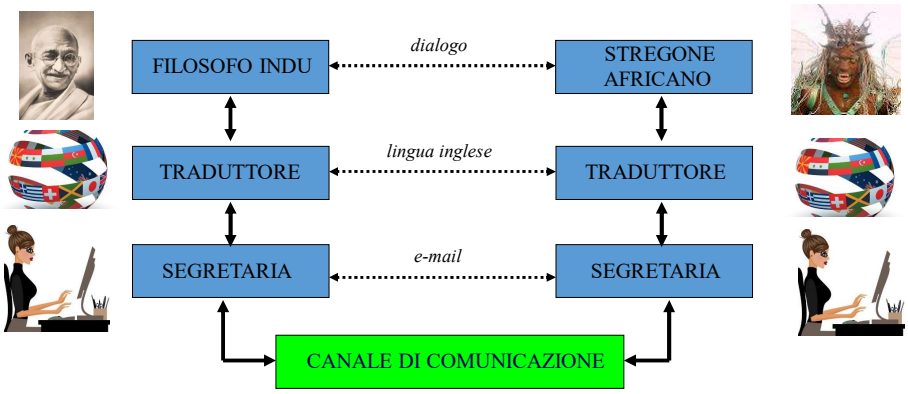
- **Standard IEEE 802 per le reti locali**
- **Architettura OSI** (Open Systems Interconnection)
- **5G**



SOFTWARE DI RETE

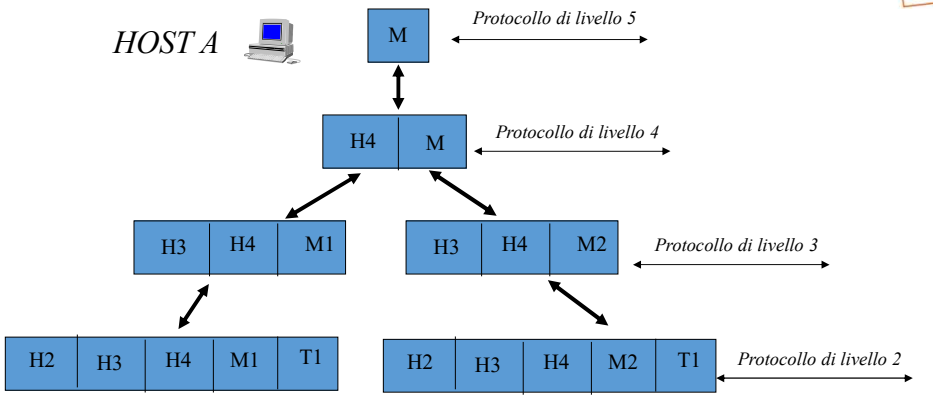
Software di rete

• Per comprendere i meccanismi basilari di funzionamento del SW di rete si può pensare (esempio) ad un filosofo indiano che vuole conversare con uno stregone africano:

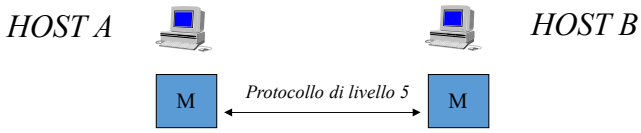


Software di rete (2)

• Nelle reti la comunicazione fra le due entità di livello superiore avviene con una modalità uguale in tutte le architetture di rete



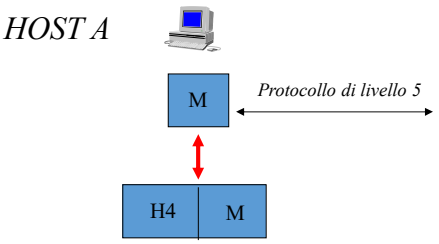
Software di rete (3)



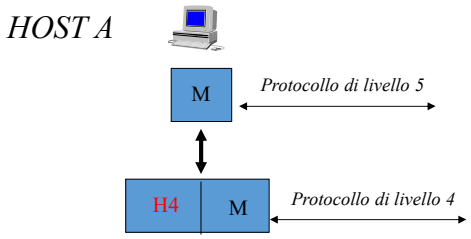
(1) Il programma applicativo (livello 5) deve mandare un messaggio ***M*** alla sua *peer entity.*

Software di rete (4)

(2) Il livello 5 consegna il messaggio ***M*** al livello 4 per la trasmissione



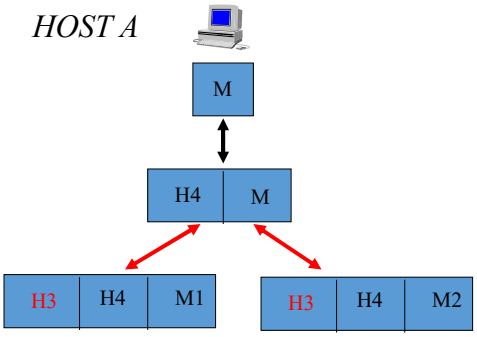
Software di rete (5)



(3) Il livello 4 aggiunge un suo ***header*** (*H4*) in testa al messaggio (talvolta si dice che il messaggio è inserito nella ***busta*** di livello 4); questo *header* contiene informazioni di controllo, tra le quali:

- Indirizzo (sorgente/destinazione)
- Numero di sequenza del messaggio
- Dimensione del messaggio

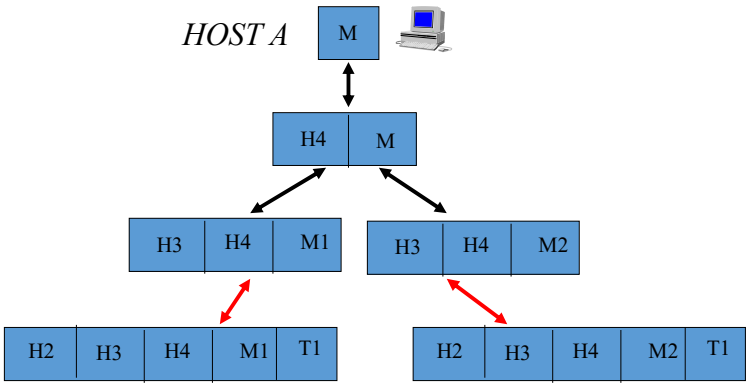
Software di rete (6)



(4) Il livello 4 consegna il risultato al livello 3

(5) Il livello 3 può trovarsi nella necessità di frammentare i dati da trasmettere in unità più piccole (***pacchetti***) a ciascuna delle quali aggiunge il suo header

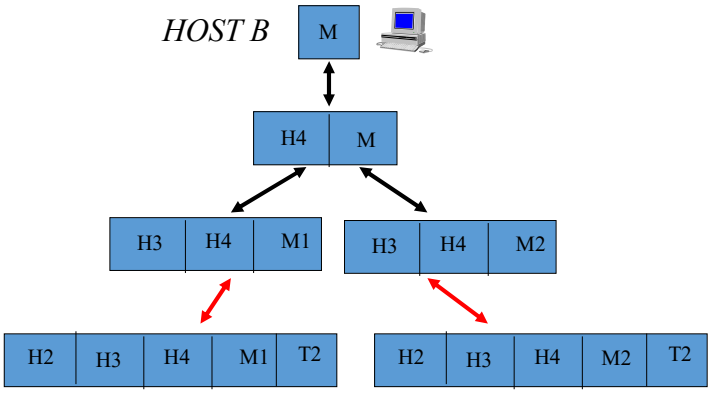
Software di rete (7)



(6) Il livello 3 passa i pacchetti al livello 2

(7) Il livello 2 aggiunge ad ogni pacchetto il proprio *header* (e un ***trailer***) e lo spedisce sul canale fisico

Software di rete (8)

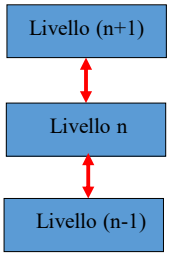


(8) Nell'host di destinazione i pacchetti fanno il percorso inverso, con ogni livello che elimina (elaborandoli) l'*header* ed il trailer di propria competenza, passando il resto al livello superiore.

INTERFACCE E SERVIZI

Interfacce e Servizi

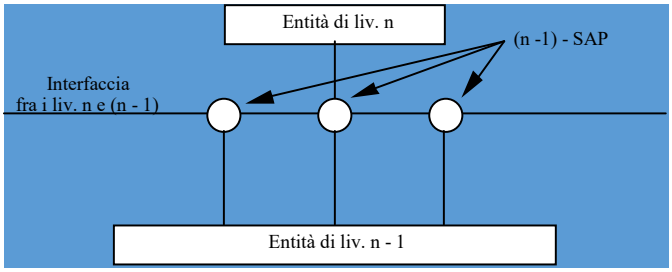
- La funzione di ogni livello è di offrire ***servizi*** al livello superiore.
- Il livello inferiore è il ***service provider***, quello superiore è il ***service user***.
- Un ***livello n*** che usufruisce dei servizi di ***livello (n-1)*** può, per mezzo di questi, a sua volta offrire al ***livello (n+1)*** i propri servizi.



Interfacce e Servizi (2)



• I servizi sono disponibili ai ***SAP (Service Access Point)***.



• I SAP del livello n, o ***n-SAP***, sono i punti di accesso nei quali il livello (*n+1*) può accedere ai servizi del livello n.

• Ogni n-SAP ha un ***indirizzo*** che lo identifica univocamente.

Interfacce e Servizi (3)



• L'informazione che passa dal livello n al livello (n-1), attraverso il (n-1)-SAP, è definita ***PDU (Protocol Data Unit) di livello n***, o ***n-PDU***.

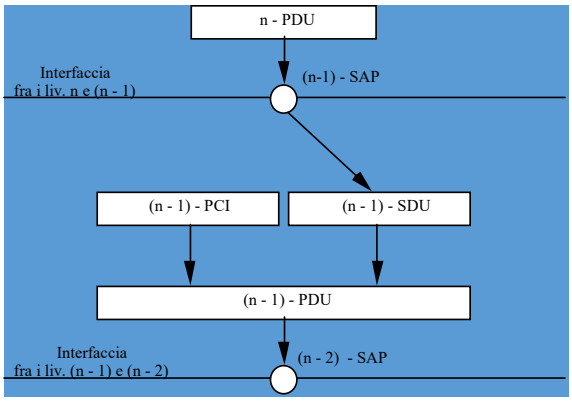
PDU	Info di controllo	DATI

• Tale informazione è così detta perché la sua parte di controllo (***header***) varia a seconda del ***layer*** (e dunque del protocollo associato) che si trova a gestirla

Interfacce e Servizi (4)



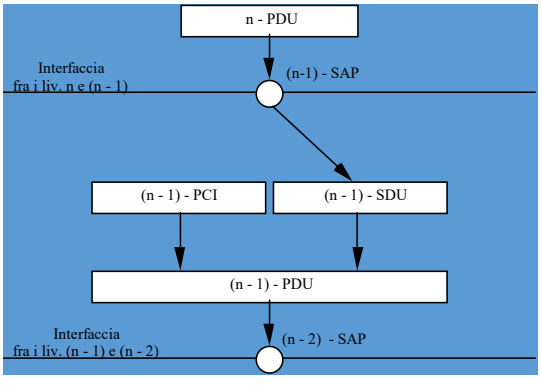
• La ***PDU*** entrando nel livello *(n-1),* diventa una ***SDU (Service Data Unit)*** di livello ***(n-1)***, o ***(n-1)-SDU***.



Interfacce e Servizi (5)



• Al livello *(n-1)* viene aggiunta alla ***(n-1)-SDU*** una ***PCI (Protocol Control Information)*** di livello ***(n-1)*** che diventa una ***(n-1)-PDU***, che verrà passata al livello (n-2) attraverso un ***(n-2)-SAP***.



Interfacce e Servizi (6)

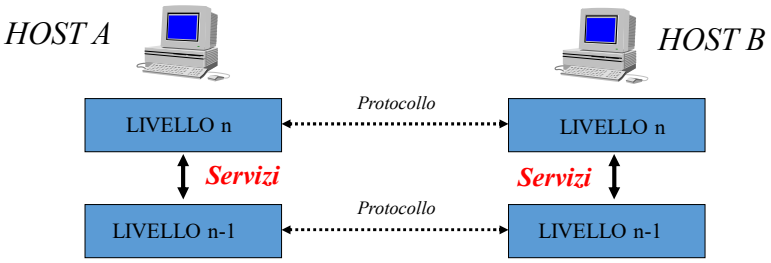
- Nomi spesso usati per i PDU:
  - ***Segmento*** (o ***TPDU, Transport PDU***) a livello di trasporto
  - ***Pacchetto*** (***packet***) a livello di rete
  - ***Trama*** (***frame***) a livello data link

Servizi vs. Protocolli

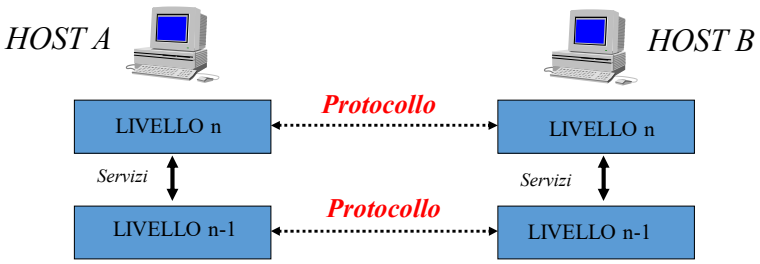
- **Servizi** e **protocolli** sono spesso *confusi*, ma sono concetti ben distinti.

SERVIZIO

- Insieme di operazioni primitive che un livello offre al livello superiore (*come tali operazioni siano implementate non riguarda il livello superiore*)



Servizi vs. Protocolli (2)



PROTOCOLLO

- Insieme di regole che governano il formato ed il significato delle informazioni (*messaggi, frame, pacchetti*) che le *peer entity* si scambiano fra loro
- Le entità usano i protocolli per implementare i propri servizi.

PARTE II

TIPOLOGIE DI SERVIZI

Tipologie di Servizi

- Un *livello n* può offrire due diversi tipi di servizi al livello superiore *(n+1)*

SERVIZI ORIENTATI ALLA CONNESSIONE

SERVIZI PRIVI DI CONNESSIONE

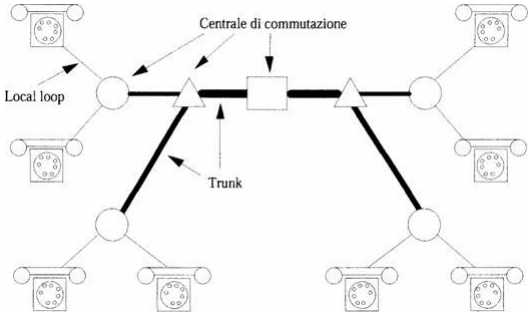
Servizi orientati alla connessione

- Sono modellati sul sistema telefonico

(1) L'utente del servizio prima stabilisce una connessione

(2) Utilizza la connessione

(3) Rilascia la connessione



Servizi privi di connessione

- Sono modellati sul sistema postale

(1) Ogni messaggio porta con se l'indirizzo completo di destinazione

(2) Ogni messaggio attraversa il sistema in maniera indipendente dagli altri

(3) *Un messaggio inviato dopo può arrivare a destinazione prima*



Affidabilità del Servizio



- Un servizio può essere ***affidabile*** (***reliable***) o non affidabile.
- Un ***servizio affidabile*** non perde mai dati, cioé assicura che tutti i dati spediti verranno consegnati al destinatario
- Richiede che il ricevente invii un ***acknowledgement o ACK*** (***conferma***) alla sorgente per ogni pacchetto ricevuto. Si introduce ovviamente ***overhead***, che in certe situazioni può non essere trascurabile.

→ Trasferimento di file

Affidabilità del Servizio (2)



- Un ***servizio non affidabile*** non offre la certezza che i dati spediti arrivino effettivamente a destinazione.
- Si noti che se un certo livello non offre nessun servizio affidabile, qualora tale funzionalità sia desiderata dovrà essere fornita da almeno uno dei livelli superiori (ciò accade spesso).

→ Posta elettronica, streaming audio-video

Affidabilità del servizio - Servizi orientati alla connessione



• ***Servizi orientati alla connessione affidabili***: trasferimento di file (non devono mancare pezzi e il file non deve essere "rimescolato")

• ***Servizi orientati alla connessione NON affidabili***: nelle trasmissioni isocrone (quali ***voce*** e ***video***) le relazioni temporali fra i bit del flusso devono essere mantenute (*è meglio qualche disturbo ogni tanto, piuttosto che interruzioni momentanee, ma avvertibili, del flusso di dati*)

➡ Voice over IP, video over IP

Affidabilità del servizio - Servizi privi di connessione



• ***Servizi privi di connessione affidabili*** (detto anche ***acknowledged datagram service***): si invia un breve messaggio e si vuole essere assolutamente sicuri che sia arrivato.

➡ Messaggistica istantanea

• ***Servizi privi di connessione NON affidabili*** (detto anche ***datagram service***)

➡ Posta elettronica pubblicitaria (*)

(*) non importa se qualche messaggio si perde.

PRIMITIVE DI SERVIZIO

Primitive di definizione del Servizio

• Un servizio di ***livello n*** è formalmente specificato da un insieme di ***primitive*** (cioé operazioni) che un entità di ***livello (n+1)*** può adoperare per accedere al servizio.

- Esse possono indicare al servizio:

  l'azione da compiere (l'informazione viaggia dal livello *n* al livello *(n-1)*)

  cosa riportare in merito ad una azione effettuata dalla peer entity di livello n (l'informazione viaggia dal livello *(n-1)* al livello *n*).

Primitive di Servizio (2)



- Tra le più importanti primitive si possono segnalare le seguenti:

request() → si chiede al servizio di fare qualcosa

indication() → si viene avvertiti, dal servizio, di qualche evento

Primitive di Servizio (3)



response() → si vuole rispondere ad un evento

confirm() → la risposta che si attendeva è arrivata

Primitive di Servizio (4)



• Per stabilire una connessione fra le *peer entity* A a B si avrà che:

Entity	Azione	Flusso informazione	Significato
A	invia una `connect.request()`	da livello n a livello (n-1)	A desidera connettersi
B	riceve una `connect.indication()`	da livello (n-1) a livello n	qualcuno vuole connettersi
B	invia una `connect.response()`	da livello n a livello (n-1)	B accetta (oppure no)
A	riceve una `connect.confirm()`	da livello (n-1) a livello n	B ha accettato (o no)

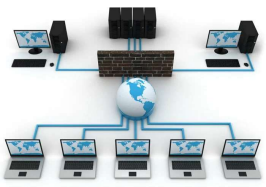
Primitive di Servizio (5)



• Le primitive hanno vari parametri (*mittente, destinatario, tipo del servizio richiesto*, ecc.), che possono essere usati dalle peer entity per negoziare le caratteristiche della connessione.

• Un servizio può essere richiesto in modalità confermata oppure non confermata.





Primitive di Servizio (6)



• Per il servizio confermato avremo:

```
request();
indication();
response();
confirm().
```

• Mentre per il servizio non confermato:

```
request();
indication().
```

- ***Connect*** è sempre confermato (ovviamente), ma altri servizi no.

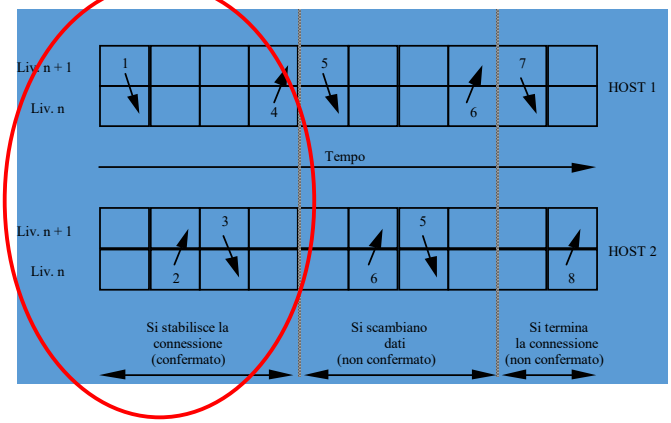
Primitive di Servizio - Esempio



Vediamo ora un esempio di servizio connection-oriented con ***8 primitive*** tra un ***host 1*** e un ***host 2***

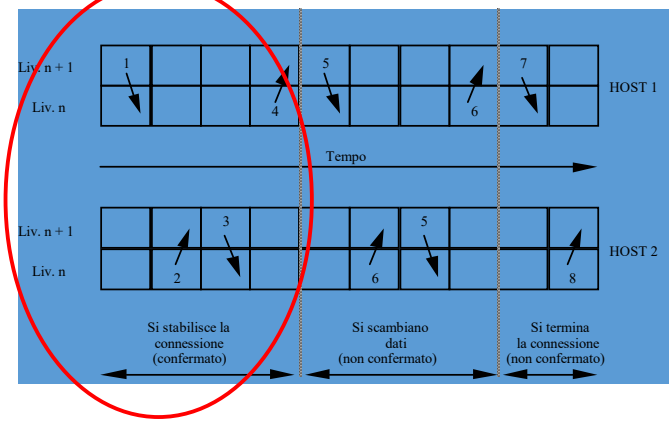
(1) `connect.request()`:
Richiesta di stabilire una connessione ***(host 1)***

(2) `connect.indication()`:
Tx un segnale all'entità chiamata ***(host 2)***



Primitive di Servizio – Esempio (2)

- Arriva la risposta dal *livello (n+1)*



(3) `connect.response()`:
Usata dall'entità chiamata per accettare o rifiutare la chiamata *(host 2)*

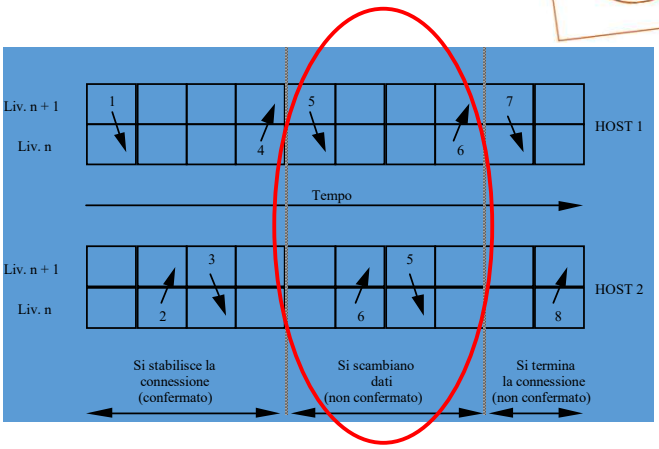
(4) `connect.confirm()`:
Comunica che la chiamata è accettata o rifiutata *(host 1)*

Primitive di Servizio – Esempio (3)

- Inizia lo scambio vero e proprio dei dati

(5) `data.request()`:
Richiede la spedizione dei dati; *(host 1)*

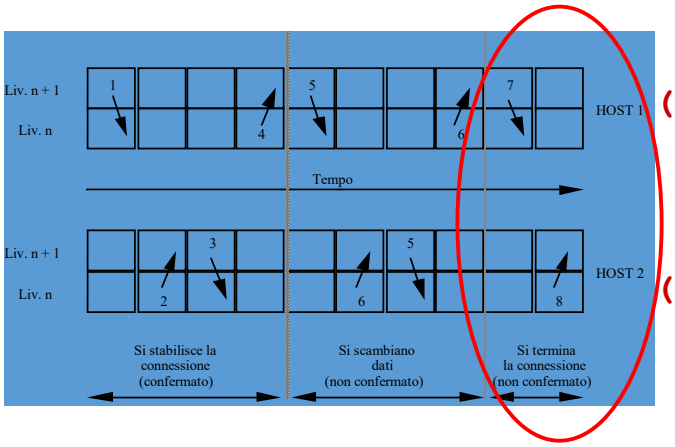
(6) `data.indication()`:
Segnala l'arrivo dei dati; *(host 2)*



Primitive di Servizio – Esempio (4)



- Si termina la connessione tra ***host 1*** e ***host 2***



(7) `disconnect.request()`: Richiede il rilascio della connessione; ***(host 1)***

(8) `disconnect.indication()`: Informa l'altra entità che vuole terminare; ***(host 2)***



PROGETTO DEI LIVELLI DI UNA ARCHITETTURA DI RETE

Aspetti di progetto dei Livelli



• Decisioni di progetto dei diversi livelli vanno prese in base a diverse problematiche esistenti.

(1) Meccanismi di identificazione di mittente e destinatario (cioé ***indirizzamento***), in ogni livello.

(2) Regole per il ***trasferimento dati*** (livelli bassi):

in una sola direzione (***simplex connection***);

in due direzioni ma non contemporaneamente (***half-duplex connection***).

in due direzioni contemporaneamente (***full-duplex connection***);

Aspetti di progetto dei Livelli (2)



(3) Meccanismi per il controllo degli ***errori di trasmissione***

Esistono diverse situazioni possibili:

rilevarli oppure no;

correggerli oppure no;

avvertire il mittente oppure no.

1000 1001
1011 0001

Aspetti di progetto dei Livelli (3)

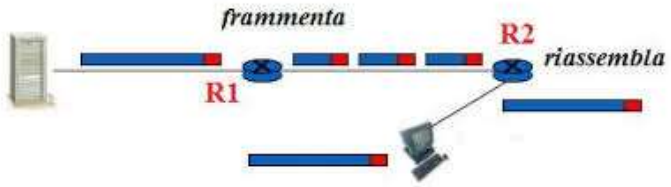
(4) Meccanismi per il mantenimento (o la ricostruzione) dell'***ordine originario*** dei dati.



(5) Meccanismi per regolare le ***velocità*** di sorgente e destinazione.

Aspetti di progetto dei Livelli (4)

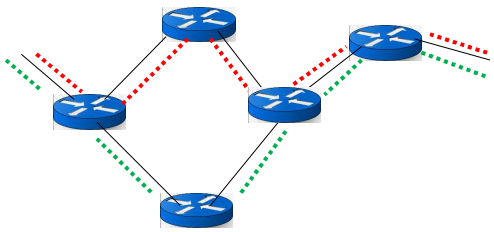
(6) Decisioni sulla ***dimensione*** (minima o massima) dei messaggi da inviare, e su come eventualmente frammentarli.



Aspetti di progetto dei Livelli (5)

(7) Meccanismi di ***multiplexing*** di varie "conversazioni" su di un'unica connessione (se stabilire la connessione è costoso).

(8) Meccanismi di ***routing*** dei messaggi se esistono più strade alternative, ed eventualmente di suddivisione di una "conversazione" su più connessioni contemporaneamente (per aumentare la velocità di trasferimento dei dati).



DOMANDE DI TEORIA

DOMANDE DI TEORIA



(a) Che cosa si intende per architettura di rete proprietaria ?

(b) Che cosa si intende per SAP e per n-PDU ?

(c) A cosa corrispondono gli indirizzi SAP nella trasmissione radio di tipo FM ?

(d) Qual è la differenza tra un servizio ed un protocollo ?

(e) Fare un esempio di comunicazione di un messaggio M tra due entità di livello n su due differenti host A e B

DOMANDE DI TEORIA (2)



(f) Qual è la differenza tra un servizio ed un protocollo ?

(g) Che cosa si intende per service provider?

(h) Che cosa si intende per service user?

(i) Qual è la differenza tra un servizio ed un protocollo ?

(l) Qual è la differenza tra un servizio orientato alla connessione ed uno privo di connessione ?

DOMANDE DI TEORIA (3)



(m) Cosa si intende per ***SAP (Service Access Point)***?

(n) Qual è la caratteristica di un indirizzo SAP?

(o) Che cosa si intende per ***PDU (Protocol Data Unit) di livello n***?

(p) Che cosa si intende per ***SDU (Service Data Unit)*** di livello ***(n-1)***?

(q) Che cosa si intende per ***PCI (Protocol Control Information)?***

(r) Qual è la differenza tra un servizio orientato alla connessione ed uno privo di connessione?

DOMANDE DI TEORIA (4)



(s) Riportare un esempio di servizio orientato alla connessione NON affidabile

(t) Descrivere il compito delle primitive confirm() e indication()

(u) Due reti forniscono un servizio affidabile orientato alla connessione. Una offre una sequenza di byte affidabile mentre l'altra fornisce una sequenza di messaggi. I due servizi offerti sono equivalenti?
Se non lo sono fornire un esempio in cui essi differiscono.

DOMANDE DI TEORIA (5)

(v) Qual e' la differenza tra un servizio confermato e uno non confermato ?
Per ciascuno dei seguenti servizi, dire se puo' essere confermato, non confermato, entrambe le cose o nessuna delle due:

(a) Apertura del collegamento

(b) TX dati

(c) Rilascio del collegamento

Bibliografia

[1] Andrew S. Tananbaum, N. Feamster, David J. Wetherall, *Reti di calcolatori. Ediz. Mylab*, (Sesta Edizione), Pearson, 2023.

[2] J. Kurose, K. Ross, *Reti di Calcolatori e Internet: un Approccio Top Down*, (Ottava Edizione), Prentice Hall, 2022.

[3] D.E. Comer, *Computer Networks and Internet*, (Sesta Edizione), Prentice Hall, 2018.

[4] Chwan-Hwa Wu and J. David Irwin, *Introduction to Computer Networks and Cybersecurity*, CRC Press, 2016.

